



Depolama

Kubernetes ve Konteyner Dünyası



Bu Konuda

- Volume yaşam döngüsü ve volume türleri
- PV, PVC ve StorageClass
- Binding, access mode ve reclaim politikaları
- Volume modu ve expansion
- CSI, snapshot ve clone

Volume Yaşam Döngüsü





Volume ve veri ömrü

- Konteynerin yazılabilir katmanı konteyner örneğine bağlıdır; yeniden başlatılan konteyner temiz bir katmanla gelir^[1]
- Pod içindeki volume, her konteynere ayrı bir `volumeMount` ile bağlanır; aynı volume'ü bağlayan konteynerler dosya paylaşabilir^[1]
- Volume verisi konteyner yeniden başlatmalarında kalır; **ephemeral** volume Pod'a, **persistent** volume ise tek bir Pod'un ömründen bağımsız bir kaynağa bağlıdır^[1,2]



Volume türleri

Tür	Asıl işi	Yaşam/yerleşim sınırı
<code>emptyDir</code>	Geçici çalışma alanı, Pod içi dosya paylaşımı	Pod ve atandığı node
<code>configMap</code> , <code>secret</code>	API verisini dosya olarak sunma	Pod'a bağlı, veri kaynağı ayrı
<code>projected</code>	Birden çok desteklenen kaynağı tek dizinde birleştirme	Kaynaklar Pod ile aynı namespace'te
<code>hostPath</code>	Node dosya veya dizinini doğrudan bağlama	O node'un yolu ve içeriği

`hostPath` kullanan aynı Pod tanımı, node'lardaki farklı dosyalar nedeniyle farklı davranabilir. ^[1]

PV, PVC ve StorageClass





PV ve PVC

- **PersistentVolume (PV):** cluster kapsamında; yönetici tarafından önceden ya da StorageClass üzerinden dinamik oluşturulur, yaşam döngüsü tek bir Pod'dan bağımsızdır^[4]
- **PersistentVolumeClaim (PVC):** namespace kapsamındaki kullanıcı isteği; kapasite, erişim modu, volume modu ve sınıf gibi gereksinimleri bildirir^[4]
- Control plane, uygun PV ile PVC'yi `claimRef` üzerinden bire bir ve tekil olarak bağlar; uygun kaynak yoksa PVC `Pending` kalır^[4]
- Pod aynı namespace'teki PVC adını kullanır; gerçek PV ve depolama sistemi ayrıntısını doğrudan seçmez^[4]

Pod -> PVC (istek) -> PV (bağlı kaynak) -> depolama sistemi



StorageClass ve provisioning

Yol	PV ne zaman oluşur?	Kim ayrıntıyı yönetir?
Statik provisioning	Yönetici önceden PV ve gerçek depolamayı hazırlar	Yönetici
Dinamik provisioning	Uygun PVC için provisioner talep üzerine volume ve PV üretir	StorageClass + provisioner

- StorageClass; `provisioner`, sürücüyeye özgü `parameters`, `reclaimPolicy`, `volumeBindingMode` ve isteğe bağlı expansion politikasını taşır^[5,6]
- PVC'de `storageClassName` verilmezse mevcut varsayılan sınıf atanabilir; varsayılan daha sonra oluşursa atama geriye dönük yapılabilir^[4,6]
- `storageClassName: ""`, sınıfsız PV ister ve dinamik provisioning'i açıkça devre dışı bırakır; alanın hiç yazılmamasıyla aynı değildir^[4]
- Birden çok varsayılan varsa Kubernetes en yeni varsayılan sınıfı kullanabilir; işletim pratiği tek varsayılan sınıfı korumaktır^[6]



Binding ve access mode

Statik eşleşmede sınıf, en az istenen kapasite, access mode, `volumeMode` ve varsa selector birlikte değerlendirilir. Dinamik provisioning'de provisioner, PVC için yeni PV üretir. ^[4]

Mod	Mount kapsamı	Kritik ayrım
<code>ReadWriteOnce</code> (RWO)	Tek node'da read-write	Aynı node'da birden çok Pod kullanabilir
<code>ReadOnlyMany</code> (ROX)	Birçok node'da read-only	Yazma korumasının tek başına garantisi değildir
<code>ReadWriteMany</code> (RWX)	Birçok node'da read-write	Backend ve sürücü desteğine bağlıdır
<code>ReadWriteOncePod</code> (RWOP)	Cluster genelinde tek Pod	Yalnız CSI; Kubernetes v1.29'dan beri kararlı

Access mode'lar PV/PVC eşleştirmesine ve bazı mount kısıtlarına katılır; `RWOP` dışındaki modlar, mount sonrası dosya sistemi izinlerinin yerine geçmez. ^[4]

Bir volume aynı anda tek bir access mode ile mount edilir; sürücünün ilan ettiği yetenek, uygulamanın ihtiyaç duyduğu paylaşım modelini karşılamalıdır. ^[4]



Reclaim ve binding politikaları

Karar	Seçenek	Sonuç
PV reclaim	Retain	PVC silinince PV Released kalır; veri temizliği ve yeniden kullanım yönetici işidir
PV reclaim	Delete	Desteklenen sağlayıcıda PV ile arka uç depolama varlığı silinir
Binding zamanı	Immediate	PVC oluşunca binding/provisioning başlar; Pod topolojisi henüz bilinmeyebilir
Binding zamanı	WaitForFirstConsumer	Pod scheduling kısıtları bilinene kadar binding/provisioning ertelenir

Dinamik PV, StorageClass `reclaimPolicy` değerini devralır; alan yazılmazsa varsayılan `Delete` 'tir. ^[4,6]



Volume Modu ve Expansion

Filesystem

- Varsayılan `volumeMode` değeridir; volume bir dizine mount edilir
- Boş bir blok aygıt destekliorsa Kubernetes ilk mount öncesi dosya sistemi oluşturabilir^[4]

Block

- Ham aygıt `volumeDevices[].devicePath` ile sunulur; `volumeMounts` kullanılmaz
- Uygulama, arada dosya sistemi olmadan ham blok aygıtını yönetebilmelidir^[4]

PVC expansion

- Yalnız büyütme yapılır; StorageClass `allowVolumeExpansion: true` olmalı ve ilgili sürücü genişletmeyi desteklemelidir^[4,6]
- Daha büyük değer PVC'de istenir; mevcut volume genişler, yeni PV oluşturulmaz; PV kapasitesini elle değiştirmek otomatik akışı bozabilir^[4]
- Tamamlanma; sürücü, depolama arka ucu ve dosya sistemi desteğine bağlıdır; PVC `capacity`, conditions ve events birlikte izlenir^[4]



CSI

```
PVC + StorageClass -> controller/provisioner -> CSI driver -> depolama arka ucu  
Pod yerleşimi      -> kubelet + node plugin -> mount veya blok aygıtı
```

- Container Storage Interface (CSI), depolama sağlayıcılarının çekirdek Kubernetes koduna eklenmeden sürücü geliştirmesine izin veren standart arayüzdür^[1]
- `StorageClass.provisioner`, dinamik sağlama için çağrılacak sürücüyü; `parameters` ise o sürücünün yorumladığı politika ayrıntılarını seçer^[6]
- Controller tarafı, sürücü destekliorsa oluşturma, silme, attach ve expansion işlemlerini yürütür; kubelet tarafındaki node plugin volume'ü hazırlayıp Pod'a mount/aygıt olarak sunar^[1]
- Access mode, topoloji, expansion, snapshot ve clone yetenekleri sürücüye göre değişir; kurulum adımı ve Kubernetes uyumluluğu sürücünün belgelerinden doğrulanır^[1]

Kaynaklar





Kaynaklar

1. Volumes: kubernetes.io/docs/concepts/storage/volumes/
2. Ephemeral Volumes: kubernetes.io/docs/concepts/storage/ephemeral-volumes/
3. Projected Volumes: kubernetes.io/docs/concepts/storage/projected-volumes/
4. Persistent Volumes: kubernetes.io/docs/concepts/storage/persistent-volumes/
5. Dynamic Volume Provisioning: kubernetes.io/docs/concepts/storage/dynamic-provisioning/
6. Storage Classes: kubernetes.io/docs/concepts/storage/storage-classes/
7. Volume Snapshots: kubernetes.io/docs/concepts/storage/volume-snapshots/
8. Volume Snapshot Classes: kubernetes.io/docs/concepts/storage/volume-snapshot-classes/
9. CSI Volume Cloning: kubernetes.io/docs/concepts/storage/volume-pvc-datasource/
10. Configure a Pod to Use a PersistentVolume for Storage: kubernetes.io/docs/tutorials/configuration/configure-persistent-volume-storage/
11. Namespaces: kubernetes.io/docs/tasks/administer-cluster/namespaces/